



Planetary Rover Path Planning Based on Improved A* Algorithm

Wei huai Wu, Xiaomei Xie^(✉), Mingzhu Wei, Nian Liu, Xin Chen,
Peng Yan, Mechali Omar, and Limei Xu

School of Aeronautics and Astronautics,
University of Electronic Science and Technology of China,
Chengdu 611731, China
mayxiezhou@uestc.edu.cn

Abstract. Developing a space rover with ability to explore robustly and autonomously the unknown outer space landscape like Moon and Mars has always been a major challenge, since the first roving remote-controlled robot, Lunokhod 1, landed on the moon. Path planning is one of important task when the rover travels a certain distance without the human control. To traverse safely on the harsh and complex planetary surface, the terrain environmental information and the ability of the roving vehicle to overcome obstacles should be taken into account. In this paper, an improved A* algorithm is proposed via introducing both the environmental characteristics (such as surface slope and surface toughness) and the rover's traversability as the constraint conditions. Comparison of the performance of the proposed A* algorithm relative to the original A* algorithm is conducted based on MATLAB platform. Numerical simulations indicate that the improved A* algorithm has shorter path and higher planning success rate.

Keywords: Path planning · Environmental characteristics · Traversability · A* algorithm

1 Introduction

The application of the first rover landed on the out space can be dated back to November 17, 1970 the Soviet Luna 17 spacecraft landed the first roving remote-controlled robot, Lunokhod 1, on the Moon. After that, several planetary rovers, such as Apollo lunar roving vehicle, China's Yutu lunar rover and Mars rovers (Sojourner, Spirit and Opportunity, and the Curiosity), have been used to explore the Moon and Mars. It is noted that all these rovers are driven by the human tele-operators. So the autonomous navigation ability of the rover is needed in the near future, while path planning is one important problem [1]. Path planning is that the rover searches for an optimal or suboptimal path from the starting position to the target position in the motion space according to some optimization criteria (such as minimum energy consumption, shortest walking route, shortest walking time, etc.), so that the lunar rover can safely and bypass all obstacles in the motion process. In the past few decades, people have applied different path planning algorithms to the launched rovers [2].

Sojourner rover is the first rover deployed by the United States on Mars. National Aeronautics and Space Administration (NASA) applied a Go To Waypoint reactive algorithm [3] similar to Tangent Bug algorithm [4]. Since only the sensor information from the rover is used, the algorithm is simple and the computation is small. Although such algorithm works well in most environments, it lacks completeness. In other words, there is no guarantee that the algorithm will halt, or that the robot will be able to find the goal even if a path exists. The algorithm also tends to produce lengthy paths in natural terrain (relative to the optimal path). It mainly used to verify the application of new technologies needed to travel longer distances (50 km) on the surface of Mars.

Rocky7 applies a sensor-based Wedge bug response algorithm that is also based on the Tangent Bug algorithm [5]. The algorithm considers the height and slope of obstacles in obstacle avoidance and uses A* algorithm to search for the local optimal path. The disadvantages of the algorithm are the limitation of the rover as a point and the difficulty of the obstacle boundary detection, which bring some difficulties to the application of the algorithm.

Maimone et al. [6] uses a rasterized network to mark the terrain on NASA's Mars Rovers Spirit and opportunity, and built a digital elevation map (DEM) based on the grid position and corresponding terrain height in order to use the D* algorithm [7, 8] in DEM map for global path planning. The algorithm not only has the ability of secondary planning, but also considers the terrain traversability factor and improves the obstacle climbing capability of the rover. But the disadvantage is that the amount of terrain data is huge, and the processor performance must be very high. Since the errors of the gyroscope and the accelerometer will increase when rover crosses complex terrain, new positioning mechanism such as visual odometer must be introduced [9].

In 2013, China successfully deployed a lunar rover called Yutu. The improved heuristic A* search algorithm has been employed to find the optimal driving path in the three-dimensional digital elevation map of the moon while considering the limitation of the automatic environment recognition ability and the limitation of the algorithm itself. For the result of path planning, a manual correction strategy is introduced to ensure the correctness and optimality of path searching [10].

The lunar surface is a highly complex and unstructured environment. The autonomous safe driving of lunar rovers on surface is not only related to the topographic information, but also related to the obstacle-crossing ability of the lunar moving vehicle itself. At the same time, the computing power of the lunar rover and path length should be taken into account to realize the autonomous navigation. Therefore, finding a path planning method with low computational complexity, traversability of simple terrain and shortest path is the first issue we should consider. In order to solve this problem, based on the A* algorithm, this paper not only introduces the environmental characteristics, but also considers the constraints of the rover when formulating the path cost function of the algorithm, and presents a path planning algorithm based on the environmental characteristics.

This paper is organized as follows. Section 2 introduces the environmental characteristics of the lunar terrain. Section 3 demonstrates the principle of the classic A* algorithm and the improved algorithm. Section 4 shows the performance of our algorithm by simulations. The conclusion is presented in the Sect. 5.

2 Environmental Characteristics of Lunar Terrain

The environmental characteristics of lunar surface is critical to plan the path of the rover during exploration. In this paper, two main features including slope and roughness of lunar surface are used to describe environmental features [11].

2.1 Lunar Slope

The slope of any point on surface of a natural terrain is defined as the angle between the fitting plane of the point and the horizontal plane [12]. It indicates the degree of inclination of the ground surface at the point, which is numerically equal to the angle θ between the unit normal vector $\vec{n}$ of the fitting plane and the unit vector $\vec{m}$ parallel to z-axis, as shown in Fig. 1. The z-axis is the height coordinate axis of the three-dimensional map. Here, the slope of the center grid point I_{ij} denoted by θ is calculated as follows.

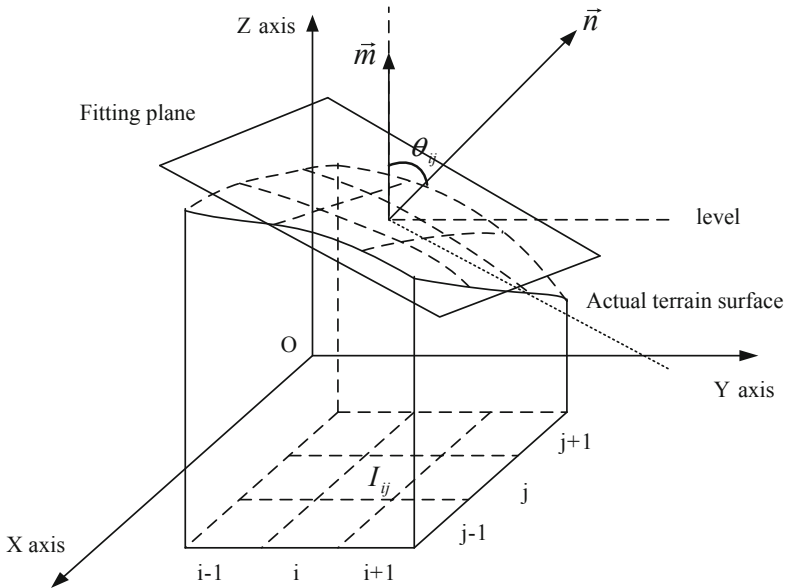


Fig. 1. Slope calculation model

In terms of Fig. 1, the analysis area is an arbitrary sub-area on the map containing 9 grids (elevation points) (x_i, y_i, z_i) , $i = 1, 2, \dots, 9$, where (x_i, y_i) is the coordinate position of the point on the grid map, and z_i is the elevation value corresponding to the point. We assume the mathematical expression of the fitting plane is

$$Ax + By + Cz + D = 0 \quad (1)$$

or

$$z = -\frac{A}{C}x - \frac{B}{C}y - \frac{D}{C} \quad (2)$$

Let $a_0 = -\frac{A}{C}$, $a_1 = -\frac{B}{C}$, $a_2 = -\frac{D}{C}$, Eq. (2) is further transformed into Eq. (3)

$$z = a_0x + a_1y + a_2 \quad (3)$$

For nine grid points in the sub-region, the above plane Eq. (3) is calculated by the nine points fitting. Hence, to determine the terms a_0 , a_1 and a_2 , Eq. (4) should be minimum.

$$S = \sum_{i=1}^9 (a_0x_i + a_1y_i + a_2 - z_i)^2 \quad (4)$$

In this context, Eq. (5) is established by considering $\frac{\partial S}{\partial a_k} = 0$, $k = 0, 1, 2$. Namely,

$$\begin{cases} a_0 \sum x_i^2 + a_1 \sum x_i y_i + a_2 \sum x_i = \sum x_i z_i \\ a_0 \sum x_i y_i + a_1 \sum y_i^2 + a_2 \sum y_i = \sum y_i z_i \\ a_0 \sum x_i + a_1 \sum y_i + a_2 n = \sum z_i \end{cases} \quad (5)$$

Solving Eq. (5) can get the values of a_0 , a_1 and a_2 . The normal vector $\vec{n}$ of the plane Eq. (3) is $\vec{n} = (a_0, a_1, -1)$.

Since the normal vector of the horizontal plane $Z = 0$ is $\vec{m} = (0, 0, 1)$, the angle between the normal vector $\vec{n}$ of the fitting plane and the plane $Z = 0$ is ϕ , as shown in Fig. 1. Thus, we have

$$\cos \theta = |\cos \langle \vec{m}, \vec{n} \rangle| = \frac{1}{\sqrt{a_0^2 + a_1^2 + 1^2}} \quad (6)$$

According to (6), the slop angle θ of the grid point I_{ij} can be obtained.

$$\theta_{ij} = \arccos \frac{1}{\sqrt{a_0^2 + a_1^2 + 1}} \quad (7)$$

2.2 Moon Surface Roughness

In actual environment, besides the steep slope, there is also uneven surface which threatens the safety of rovers. Here, roughness is used to measure the safety of the lunar rover passing through uneven surface. Generally speaking, the roughness of a terrain is

defined as the ratio of surface area S_{surface} of a surface unit to its projected area S_{flat} on a horizontal plane [13].

$$R = S_{\text{surface}} / S_{\text{flat}} \quad (8)$$

In practice, the roughness of the terrain can be expressed by fitted the mean deviation. However, because a grid has a small size and its fitting plane does not reflect the actual topographical features, a window area (3×3 grids) is used to calculate the fitted mean deviation. It is easy to know that the distance from each elevation point in the window area (nine grids) to this fitting plane is

$$d_i = \frac{|a_0x_i + a_1y_i + a_2 - z_i|}{\sqrt{a_0^2 + a_1^2 + (-1)^2}}, \quad i = 1, \dots, 9 \quad (9)$$

Based on (9), the average of distances from 9 grid points to the fitting plane is:

$$\bar{D} = \frac{1}{9} \sum_{i=1}^9 |d_i| = \frac{1}{9} \sum_{i=1}^9 \frac{|a_0x_i + a_1y_i + a_2 - z_i|}{\sqrt{a_0^2 + a_1^2 + (-1)^2}} \quad (10)$$

The fitted mean deviation $\bar{D}_{ij}$ of each grid in the window area is equal to the average distance of 9 grids to the fit plane.

$$\bar{D}_{ij} = \bar{D} = \frac{1}{9} \sum_{i=1}^9 |d_i| = \frac{1}{9} \sum_{i=1}^9 \frac{|a_0x_i + a_1y_i + a_2 - z_i|}{\sqrt{a_0^2 + a_1^2 + (-1)^2}} \quad (11)$$

3 Proposed Improved A* Algorithm

The A* algorithm was originally proposed by Hart et al. based on the Dijkstras algorithm in 1968 [14]. It combines the advantages of Dijkstras algorithm and Best-First Search (BFS) and is the most popular one of the graph search algorithms.

3.1 Principle of Classic A* Algorithm

The principle of the classic A* algorithm is to design a cost estimation function (12).

$$f(n) = g(n) + h(n) \quad (12)$$

The evaluation function $f(n)$ is the estimated value of the minimum cost path from the starting grid to grid n and then to the target grid. The function $g(n)$ is the actual cost of elapsed path from the starting grid to the grid n . The heuristic function $h(n)$ is the estimated cost of the possible optimal path from the grid n to the target grid.

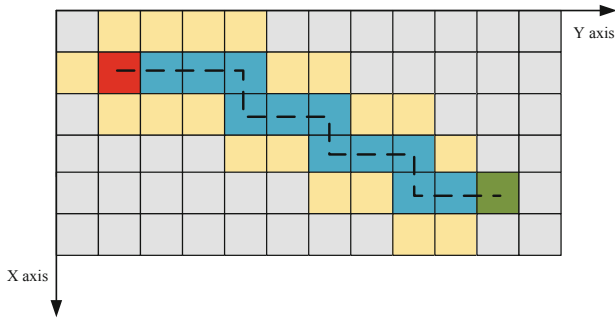


Fig. 2. A path for manhattan distance (Color figure online)

It is noted that a good heuristic function $h(n)$ is important when determining the minimum cost estimate from any grid n to the target grid. The standard heuristic function is based on the Manhattan distance. Referring to Fig. 2, red grid is the grid n which two-dimensional coordinates are (x_n, y_n) , green grid is the target grid and its corresponding two-dimensional coordinates are (x_t, y_t) . The coordinate value of each grid is an integer. Assuming the pure distance movement cost between two adjacent grids is L m (not considering terrain and diagonal movement, shown in Fig. 2, then the estimated cost $h(n)$ from grid n to target grid can be expressed as Eq. (13).

$$h(n) = L * (abs(x_t - x_n) + abs(y_t - y_n)) \quad (13)$$

3.2 Improved A* Algorithm

The classic A* algorithm uses Manhattan distance to calculate the movement between adjacent grids. However, the planetary rover can move in the diagonal direction on the map. Thus, we use a different heuristic function. Referring to Fig. 3, red grid is the grid n defined by the two-dimensional coordinates (x_n, y_n) , green grid is the target grid and its two-dimensional coordinates are (x_t, y_t) . The coordinate value of each grid is an integer. We assume that the movement cost between two adjacent diagonal grids can be simply $\sqrt{2}L$ m, then the estimated cost $h(n)$ from grid n to target grid can be expressed as Eq. (16).

According to mathematical knowledge, the following (14), (15), (16) can be obtained.

$$h_diagonal(n) = \min(abs(x_t - x_n), abs(y_t - y_n)) \quad (14)$$

$$h_straight(n) = (abs(x_t - x_n) + abs(y_t - y_n)) \quad (15)$$

$$h(n) = \sqrt{2}L * h_diagonal(n) + L * (h_straight(n) - 2 * h_diagonal(n)) \quad (16)$$

Where, $h_diagonal(n)$: the number of steps that can be moved along a diagonal line; $h_straight(n)$: the number of steps in Manhattan distance. All slash steps are multiplied

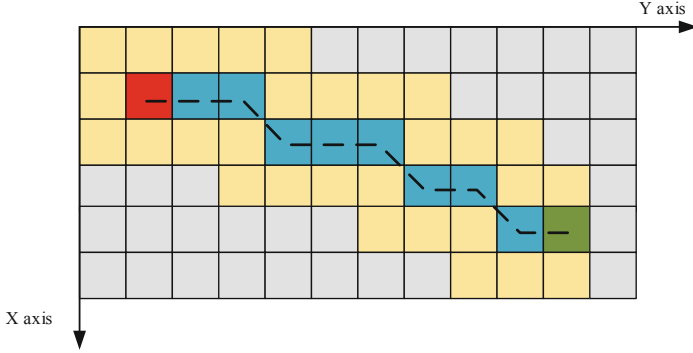


Fig. 3. A path for diagonal distance (Color figure online)

by $\sqrt{2}L$, and all line steps (Manhattan distance steps minus 2 times slash steps) are multiplied by L .

Moreover, for classic A* algorithm, we simply divide the grids in the map into obstacles (not passable) and barrier-free (passable), without considering that the lunar rover has certain obstacle-crossing ability and can climb over some relatively simple obstacles. Taking the Yutu as an example, it can generally climb a slope below 20° or a step below 20 cm safely. The trafficability of the robot at the grid point I_{ij} can be obtained by judging whether the grid point meets the safety requirements. That is, whether the gradient θ_{ij} of the grid is less than the maximum safety gradient $\theta_{\max}$, and whether the roughness $\bar{D}_{ij}$ of the grid is less than the maximum safety roughness $\bar{D}_{\max}$. If they are less than or equal to safety value, the grid satisfies the condition of passability; otherwise, the grid is directly determined to be unsafe [15].

In the actual test, we get many paths satisfying the conditions of accessibility. How to choose an optimal path among these paths is a problem worth studying. Here we may take the shortest path length as the optimization criterion. Assume that the path from grid n to the target grid is divided into e straight steps and f oblique steps. For terrain slope, the additional length of traveling through the grid are W . Similarly, for terrain roughness, the additional length of traveling through the grid are E , then the estimated cost $h(n)$ from grid n to target grid can be expressed as Eq. (17).

$$h(n) = \sqrt{2}L * h_diagonal(n) + L * (h_straight(n) - 2 * h_diagonal(n)) + W + E \quad (17)$$

Where, $W = \sum_{i=1}^f L * \left(\frac{1.4}{\cos \theta_i} - 1.4 \right) + \sum_{k=1}^e L * \left(\frac{1}{\cos \theta_k} - 1 \right)$, and $E = \sum_{i=1}^f \sqrt{2} * \bar{D}_i + \sum_{k=1}^e \bar{D}_k$.

The pseudocode of the improved A* algorithm is provided in Table 1.

Table 1. The pseudocode of the improved A* algorithm**Algorithms 1** Improved A* algorithm**Require:** $\theta_{\max}$: Maximum safety gradient**Require:** $\bar{D}_{\max}$: Maximum safety roughness**Require:** (x_0, y_0, z_0) : The coordinates of the starting grid n_{start} **Require:** (x_t, y_t, z_t) : The coordinates of the target grid n_{target} 1:/* Search for a path from start grid n_{start} to target grid n_{target} and return the result */2: **function** Evaluation(n)3 $h_diagonal(n) = \min(abs(x_t - x_n), abs(y_t - y_n))$ 4: $h_straight(n) = (abs(x_t - x_n) + abs(y_t - y_n))$ 5: $f = h_diagonal(n) = \min(abs(x_t - x_n), abs(y_t - y_n))$ 6: $e = abs(x_t - x_n) + abs(y_t - y_n) - 2 * \min(abs(x_t - x_n), abs(y_t - y_n))$ 7: $W = \sum_{i=1}^f L * \left(\frac{1.4}{\cos \theta_i} - 1.4 \right) + \sum_{k=1}^e L * \left(\frac{1}{\cos \theta_k} - 1 \right)$ 8: $E = \sum_{i=1}^f \sqrt{2} * \bar{D}_i + \sum_{k=1}^e \bar{D}_k$ 9: $h(n) = \sqrt{2} L * h_diagonal(n) + L * (h_straight(n) - 2 * h_diagonal(n))$
 $+ W + E$ 10: $f(n) = g(n) + h(n)$ 11: **return** $f(n)$ 12: **end function**13: **function** Traversability(n)14: if $\theta_n > \theta_{\max}$ OR $\bar{D}_n > \bar{D}_{\max}$ then15: Traversability(n) < 016: **ignore this grid n, grid n is not safe**

17: else

18: Traversability(n) > 019: **end if**20: **return** Traversability(n)21: **end function**


```

22: function UpdateState( $n, n'$ )
     $c(n, n') = \sqrt{2} L * h\_diagonal(n, n')$ 
23:      $+ L * (h\_straight(n, n') - 2 * h\_diagonal(n, n'))$ 
         $+ W(n, n') + E(n, n')$ 
24:     if Traversability( $n'$ ) < 0 OR  $n' \in \text{CLOSED}$  then
25:         Ignore this  $n'$ 
26:     else
27:         if  $n' \in \text{OPEN}$  then
28:             if  $g(n') > g(n) + c(n, n')$  then
29:                  $parent(n') = n$ 
30:                  $g(n') = g(n) + c(n, n')$ 
31:                 Evaluation( $n'$ )
32:             else
33:                 ignore this  $n'$ 
34:             end if
35:         else
36:             OPEN.Insert( $n'$ , Evaluation( $n'$ ))
37:              $parent(n') = n$ 
38:              $g(n') = g(n) + c(n, n')$ 
39:         end if
40:     end if
41: end function

42: function Main()
43:      $g(n_{start}) = 0$ 
44:     OPEN =  $\emptyset$ 
45:     CLOSED =  $\emptyset$ 
46:     OPEN.Insert( $n_{start}$ , Evaluation( $n_{start}$ ))
47:     while OPEN  $\neq \emptyset$  do
48:          $n = \text{OPEN.minEvaluation}(n)$ 
49:         CLOSED = CLOSED  $\cup n$ 
50:         if  $n = n_{target}$  then
51:             return "Find the optimal path"
52:         else
53:              $n'$  are adjacent 8 grids of  $n$ 
54:             for all  $n'$  do
55:                 updateState( $n, n'$ )
56:             end if
57:         end while
58:         return "Path planning failed"
59: end function

```

4 Simulation Experiments

Because there is no detailed digital elevation map (DEM) of local lunar surface, the interpolation method is used in MATLAB R2018A to build the grid map similar to the planetary surface environment. The map consists of three hillsides, four deep pits and rough ground. The classic A* algorithm and the improved A* algorithm are used to plan the path on the map. The experimental results are shown in Fig. 4. The size of the map is 100 m $\times$ 100 m, the size of unit grid is 1 m $\times$ 1 m. The blue points in the map are the unsafe grids (the slope is greater than 20° or the roughness is greater than 0.2 m) that the lunar rover cannot pass through. The starting point and target point on the map are produced randomly. For example, the starting point coordinate and the target point coordinate illustrated in Fig. 4 are (5 m, 5 m) and (59 m, 82 m) respectively.

As shown in Fig. 4, the red solid path planned by the improved A* algorithm is not only shorter, but also traverses some safe terrain obstacles. However, the black-dotted path planned by the classic A* algorithm passes through dangerous terrain that the lunar rover cannot cross. It means such path planning has actually failed and further proves that the improved A* algorithm is more suitable for the complex unstructured environment of the lunar surface.

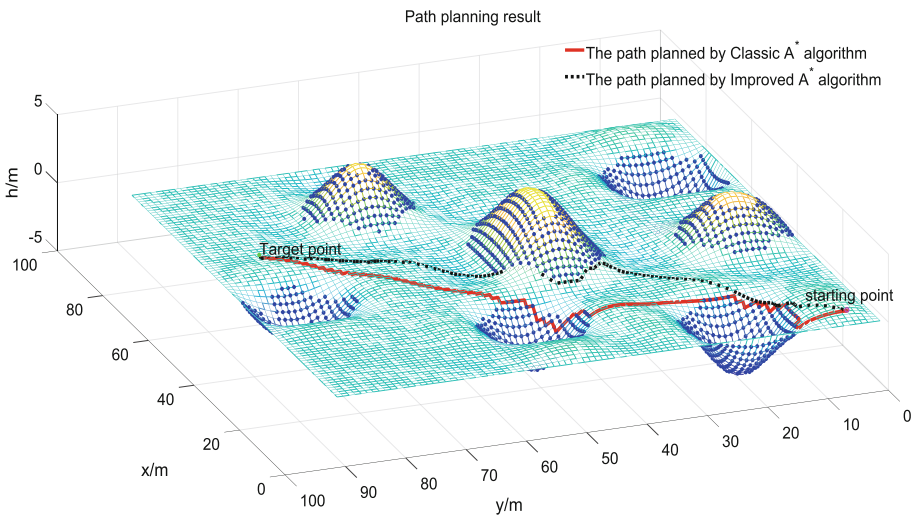


Fig. 4. Path planning results of two algorithms on the same map (Color figure online)

Changing the starting point and target point on the map and running both the classic and improved A* to get a new couple of paths, 50 different experiments are repeatedly done. The numerical simulation results are summarized in Figs. 5 and 6 respectively. Figure 5 presents the length of each path planned by each algorithm. Figure 6 illustrates that whether the planning path is successful for each experiment for each algorithm.

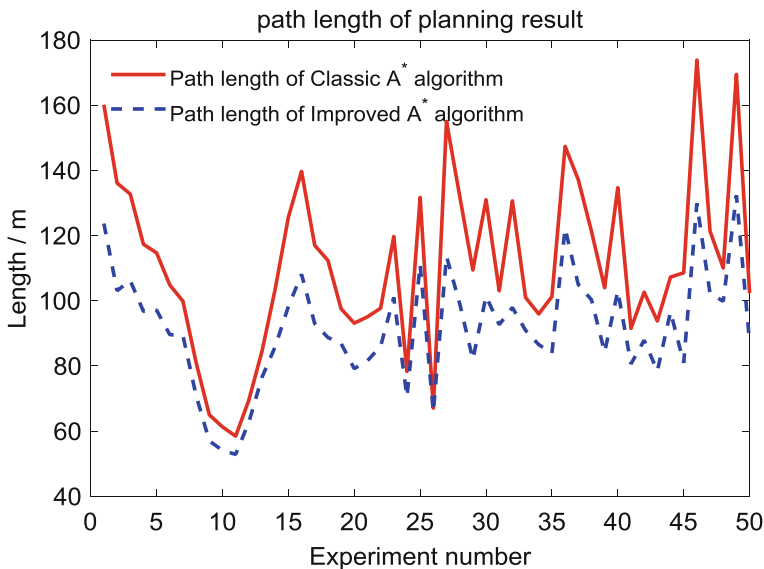


Fig. 5. 50 sets of experimental data for the path lengths of the two algorithms.

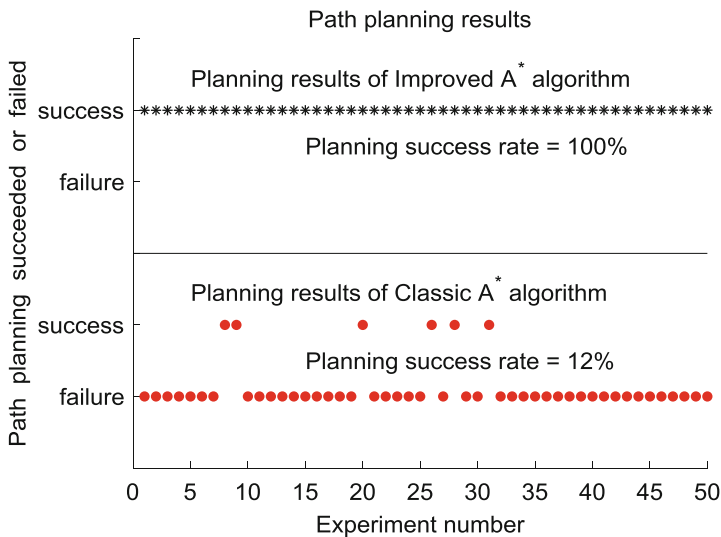


Fig. 6. 50 sets of experimental data for the planning results of the two algorithms.

The numerical results shown in Fig. 5 indicate that the path length of the improved A* algorithm for most of the 50 experiments is shorter than the path length of the classic A* algorithm. It can be seen from Fig. 6, the improved A* algorithm is definitely able to successfully plan a safe path, while the classic A* algorithm tends to pass some dangerous terrain and results in a low planning success rate.

5 Conclusion

A* algorithm is often used to solve the shortest path problem because of its simple principle and low computation. It uses the heuristic function to reduce the search space and improve the search efficiency, so the heuristic function plays a key role in the A* algorithm. To traverse safely on the harsh and complex planetary surface, the terrain environmental information and the ability of the roving vehicle to overcome obstacles should be taken into account. In this paper, we introduce the obstacle-crossing ability of lunar rover, lunar slope and moon surface roughness into the heuristic function of the algorithm, and allow the lunar rover to move obliquely which is more like to the actual movement of the lunar rover on the lunar surface. The simulation results show that the improved A* algorithm has higher planning success rate and shorter path than the classic A* algorithm for unstructured and complex lunar environment.

Acknowledgment. The author would like to acknowledge the support from the Advanced Research Project of Manned Space under Grant No. 0603(17700630), the National Natural Science Foundation of China under Grant No. 61803075, the Fundamental Research Funds for the Central Universities under Grant No. ZYGX2018KYQD211.

References

1. Wong, C., Yang, E., Yan, X., Gu, D.: Adaptive and intelligent navigation of autonomous planetary rovers—a survey. In: 2017 NASA/ESA Conference on Adaptive Hardware and Systems (AHS), Pasadena, CA, pp. 237–244 (2017)
2. Grotzinger, J.P., et al.: Mars science laboratory mission and science investigation. *Space Sci. Rev.* **170**(1–4), 5–56 (2012)
3. Wilcox, B., Nguyen, T.: Sojourner on Mars and lessons learned for future planetary rovers. SAE Technical Paper 10.4271/981695 (1998)
4. Laubach, S.L., Burdick, J., Matthies, L.: An autonomous path planner implemented on the Rocky 7 prototype microrover. In: Proceedings of 1998 IEEE International Conference on Robotics and Automation (Cat. No. 98CH36146), Leuven, Belgium (1998)
5. Laubach, S.L., Burdick, J.W.: An autonomous sensor-based path-planner for planetary microrovers. In: Proceedings of 1999 IEEE International Conference on Robotics and Automation (Cat. No. 99CH36288C), Detroit, MI, USA (1999)
6. Maimone, M., Biesiadecki, J.J., Tunstel, E., Cheng, Y., Leger, C.: Chapter 3: Surface navigation and mobility intelligence on the Mars exploration rovers (2006)
7. Maimone, M.W., Leger, P.C., Biesiadecki, J.J.: Overview of the mars exploration rovers' autonomous mobility and vision capabilities. In: Proceedings of 2007 IEEE International Conference on Robotics and Automation (2007)
8. Stentz, A.: The D* algorithm for real-time planning of optimal traverses. CMU Technical report CMU-RI-TR-94–37 (1994)
9. Carsten, J., Rankin, A., Ferguson, D., Stentz, A.: Global path planning on board the Mars exploration rovers. In: 2007 IEEE Aerospace Conference, Big Sky, MT (2007)
10. Wu, W., Zhou, J., Wang, B., Liu, C.: Key techniques in the teleoperation of the Yutu no. 3 “Yutu” lunar rover. *Sci. China Inf. Sci.* (2014)
11. Chavez-Garcia, R.O., Guzzi, J., Gambardella, L.M., Giusti, A.: Learning ground traversability from simulations. *IEEE Robot. Autom. Lett.* **3**(3), 1695–1702 (2018)

12. Brooks, R.: A robust layered control system for a mobile robot. *IEEE J. Robot. Autom.* **2**(1), 14–23 (1986)
13. Moore, I.D., Grayson, R.B., Ladson, A.R.: Digital terrain modeling: a review of hydrological, geomorphological, and biological applications. *Hydrol. Process* **5**, 3–30 (1991)
14. Hart, P.E., Nilsson, N.J., Raphael, B.: A formal basis for the heuristic determination of minimum cost paths. *IEEE Trans. Syst. Sci. Cybern.* **SSC4** **4**(4), 100–107 (1968)
15. Hadsell, R., Scoffier, M., Muller, U., LeCun, Y., Sermanet, P.: Mapping and planning under uncertainty in mobile robots with long-range perception. In: *Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 1–6. IEEE (2008)